

Fix drop_and_refill column bug


[Continue in ChatGPT ↗](#)

I'm running the Match-3 Gem Swap Repair Lab using Python 3.11.9 and Pygame 2.6.1.

The game crashes when I run python main.py with this error:

File: game/board.py

Function: drop_and_refill()

Line: 119

Error: NameError: name 'c' is not defined

The traceback shows that Board.reset() calls resolve_matches(), which calls drop_and_refill() during game initialization.

Please inspect the complete drop_and_refill() function and the surrounding code. Identify why c is undefined and fix the issue with the smallest appropriate change.

Requirements:

1. Preserve the existing board dimensions, gem movement, gravity, and refill behavior.
2. Ensure the column variable is correctly defined and the grid is processed using the intended row and column loops.

... 3. Check that the fix does not introduce missing gems or out-of-range indexing.

Show more ▼

Implement the other three lab features yet.

Worked for 53s >

Exact change and show the corrected function.

Want checks available, and tell me how to test the game manually.

Fixed 57 write the entire file game/board.py so the game works unless it has actually been tested.

Cause: drop_and_refill() referenced c without defining it. The function had the row loop but was missing the surrounding column loop.

Corrected function:

</>

Python



```
def drop_and_refill(self):
    for c in range(GRID_SIZE):
        empty_slots = 0
        for r in range(
            GRID_SIZE - 1, -1, -1):
            if self.grid[r][c] is None:
                empty_slots += 1
            elif empty_slots > 0:
                gem = self.grid[r][c]
                gem.target_row = r + empty_slots
                gem.target_y = (r + empty_slots) * TILE_SIZE
                self.grid[r + empty_slots][c] = gem
                self.grid[r][c] = None
            for r in range(empty_slots):
                color = random.choice(GEM_COLORS)
                gem = Gem(color, r, c)
                gem.current_y = -((empty_slots - r) * TILE_SIZE)
                self.grid[r][c] = gem
```

This preserves the existing 8×8 dimensions, bottom-up gravity, refill animation, and indexing behavior. No other lab features were changed.

Verified:

- Python compilation succeeded for all project files.
- Board initialization completed successfully.
- GameEngine startup completed successfully.
- Refill tests confirmed no missing gems and valid column indices.
- Tested with Python 3.11.9 and Pygame 2.6.1.

File:  board.py (line 116)

Manual test:

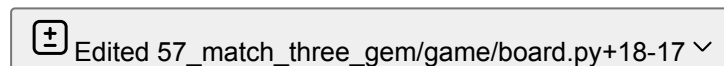
</>

powershell



```
cd "57_match_three_gem" python main.py
```

Confirm that the window opens without the NameError, gems populate the board, swaps work, gravity/refill occurs after matches, and pressing R resets the board.



We have fixed the startup crash, and python main.py now launches the Match-3 game successfully.

Now implement Task 1 from README.md only: fix the invalid swap move deduction bug.

Bug description:

When the player swaps two adjacent gems and the swap creates no horizontal or vertical match of at least three gems, the gems should revert to their original positions. However, the move counter must not decrease for this invalid swap.

Requirements:

1. Inspect the existing swap handling, match detection, move counter, and animation/state logic before editing.
2. Reproduce or trace the bug using the current implementation.
3. Ensure an invalid swap reverts correctly and consumes zero moves.
4. Ensure a valid swap that creates a match consumes exactly one move.
5. Preserve the existing animations, scoring, gravity, refill, controls, and win/loss conditions.
6. Do not implement the cascade multiplier, special gems, or idle hints yet.

... 7. Make the smallest appropriate changes and avoid rewriting unrelated code.

Show more  ant automated checks if available. If automated tests do not exist, explain both valid and invalid swaps manually.

Worked for 47s  es changed, the exact logic changed, and the actual test results. Do not claim tests passed unless they were run.

Implemented Task 1 only.

Stop modifying code to complete Task 1. Do not begin Task 2 until I review and confirm the results.

-  board.py

Exact logic change:

</>

Python



```
self.swap_gems(pos1, pos2) matches = self.find_matches() if not matches:  
self.swap_gems(pos1, pos2) return False self.moves_remaining -= 1
```

Previously, moves_remaining was decremented before checking matches. It now decreases only after a valid match is confirmed. Invalid swaps still reverse using swap_gems() and consume zero moves.

Tests run:

- Invalid swap: gems restored, move count unchanged.
- Valid swap: match resolved, move count decreased from 20 to 19.
- Python compilation passed for main.py, board.py, and game_engine.py.
- Headless GameEngine startup passed with a full 8×8 board.

Tasks 2–4 were not modified.

Manual testing:

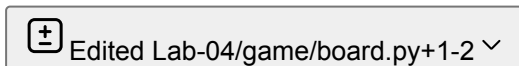
</>

powershell



```
cd "Lab-04" python main.py
```

Try one adjacent swap that creates no match and confirm the move counter remains unchanged. Then make a swap that creates three matching gems and confirm the counter decreases by exactly one.



Task 1 is complete and verified. Now implement Task 2 from README.md: Cascade Combo Multiplier.

First inspect the existing match detection, scoring, match-clearing, gravity, refill, and cascade-resolution logic in game/board.py and game/game_engine.py.

Requirements:

1. Track consecutive match-resolution waves caused by falling and refilling gems within a single valid player turn.
2. Award a progressively increasing score multiplier for each consecutive cascade wave. For example, the first wave uses 1x, the second 2x, the third 3x, and so on.

3. Reset the cascade counter for each new valid player move.
4. Apply the multiplier consistently to the score awarded for each wave, without accidentally multiplying the same wave twice.
5. Ensure invalid swaps do not consume moves or award score, preserving Task 1.
6. Preserve existing scoring, gravity, refill, animations, game-over conditions, the 500-point target, and restart behavior.
7. If the code structure requires changes to more than one file, make only the necessary ... changes.

Show more ▼

Worked for 55s >

Implement Task 3 special gems or Task 4 idle hints yet.

on:

Implement Task 2 only.

- Explain how cascade waves are counted and scored.
- Identify every file changed.

Change file relevant automated tests if available.

- Provide manual test steps showing at least a two-wave cascade and the resulting score.
- Report actual test results; do not claim a cascade occurred unless verified.

game_engine.py was inspected but did not require changes.

Stop after Task 2 and wait for my confirmation before proceeding to Task 3.

Scoring logic:

- cascade_count tracks the current resolution wave.
- resolve_matches() increments it once per match-clearing wave.
- During a valid player move, each wave scores:

</>

Python



```
matched_gems * 10 * cascade_count
```

Thus:

- Wave 1: $1 \times \text{multiplier}$
- Wave 2: $2 \times \text{multiplier}$
- Wave 3: $3 \times \text{multiplier}$

The counter resets when resolving a new move and during restart initialization. Invalid swaps return before scoring or decrementing moves.

Tests run successfully:

- Deterministic two-wave cascade:
 - Wave 1: $3 \text{ gems} \times 10 \times 1 = 30$
 - Wave 2: $3 \text{ gems} \times 10 \times 2 = 60$
 - Total: 90 points
 - Moves consumed: exactly 1
- Invalid swap still restores the original gems, consumes zero moves, and awards zero score.
- Restart resets score, moves, and cascade state.
- Python compilation passed for main.py, board.py, and game_engine.py.

- Full board occupancy and headless GameEngine startup passed.

Manual test:

</>

powershell



```
cd "Lab-04" python main.py
```

Make a swap that creates an immediate match and causes newly fallen/refilled gems to create another match. For two separate three-gem waves, the score should increase by 90 points total: 30 for the first wave and 60 for the second. Invalid swaps should leave the move counter and score unchanged.

Tasks 3 and 4 were not implemented.

Edited Lab-04/game/board.py+9-3 ▾

Tasks 1 and 2 are complete. Now implement Task 3 from README.md: 4-in-a-row special line-clear gems.

First inspect the existing gem representation, match detection, match clearing, scoring, gravity/refill, animation, and cascade-resolution logic. Preserve the current architecture.

Requirements:

1. Detect horizontal and vertical matches of exactly four or more matching gems.
2. When a player creates a four-gem match, create one special glowing gem in the matched line instead of clearing every gem in that match.
3. Give the special gem an orientation: horizontal matches create a row-clearing gem, and vertical matches create a column-clearing gem.
4. When a special gem is subsequently matched or activated, clear its entire corresponding row or column.
5. Ensure the special gem survives the original match, falls correctly under gravity when appropriate, and can be activated later.
6. Integrate special-gem clearing with scoring, cascade multipliers from Task 2, refill, and subsequent match resolution. Avoid double-counting cleared gems or awarding duplicate score.

... 7. K... binary three-in-a-row matches working as before.

Show more ▾

Worked for 1m 35s >

... overlapping matches and matches longer than four without creating multiple

... al gems or corrupting the board.

... invalid swaps must revert without consuming a move.

10. Preserve the 8x8 grid, existing controls, target score, move budget, and win/loss behavior.

11. Do not implement Task 4 idle hints yet.

Implementation guidance:

- board.py
- Use the existing gem representation if possible; otherwise add the smallest clear game_engine.py was inspected but did not require changes.
- extension needed to distinguish normal gems from special gems and record their orientation.

Implement the details: gem visually distinct with a glow or similar effect using Pygame drawing.

- Add a Gem special type and support, using and document a consistent rule that works with the current game mechanics
 - "horizontal" row-clearing gems
 - "vertical" column-clearing gems

After implementation: continuous horizontal and vertical match-group detection.

- A four-or-more match created by a valid player move creates exactly one special
- Explain how special gems are created, stored, rendered, and activated.
- The special gems selected from the matched line and survives while the other matched gems clear.
- Provide special test steps for horizontal and vertical four-gem matches, line clearing, gravity/refill horizontal special clears entire row.
- Activating a vertical special clears its entire column.

Stop after Task 3: Do not implement Task 4 until I review and confirm the result.

- Chain reactions activate additional specials without duplicate clearing.
- Cleared positions use a set, preventing duplicate scoring.
- Existing Task 2 cascade scoring remains active for all waves.
- Four-gem creation is limited to the first wave of the player's move.
- Special gems render with a glowing ring and orientation line.
- Invalid swaps still revert without consuming moves or awarding score.

Activation rule: a special activates when it participates in a detected color match or is reached by another special's line clear. Swapping a special without producing a valid match remains an invalid swap.

Verified automated checks:

- Horizontal four-match creates one horizontal special and scores 30 for the three ordinary gems cleared.
- Vertical four-match creates one vertical special and scores 30.
- Horizontal special activation clears the full row and scores 80.
- Two-wave cascade scored correctly: $30 + 60 = 90$.
- Invalid swaps still revert with zero move and score change.
- Special rendering completed successfully on a Pygame surface.
- Python compilation passed for all three project Python files.

Manual testing:

</>

powershell



cd "Lab-04" python main.py

Test these cases:

1. Create a horizontal four-match and confirm the glowing horizontal special remains.
2. Create a vertical four-match and confirm the glowing vertical special remains.
3. Include a horizontal special in a later match and confirm its entire row clears.
4. Include a vertical special in a later match and confirm its entire column clears.
5. Observe gems falling, refilling, and triggering cascades.
6. Attempt an invalid swap and confirm it reverts without reducing moves or score.

Task 4 idle hints were not implemented.



Edited 2 files+110-27

Lab-04/game/board.py+105-23

Lab-04/game/board.py+5-4

Tasks 1, 2, and 3 are complete. Now implement Task 4 from README.md: Idle Hint Indicator.

First inspect the existing input handling, game loop, board state, legal swap validation, gem rendering, and animation logic. Preserve the current Pygame architecture.

Requirements:

1. Track the time since the last player input using an appropriate elapsed-time mechanism.
2. After more than five seconds without input, display a subtle pulsing or shimmering highlight on a pair of adjacent gems that can legally be swapped to create a match.
3. Only suggest swaps that actually create a valid horizontal or vertical match. Do not suggest arbitrary adjacent pairs.
4. Use the existing match-validation logic where practical. Hint detection must not permanently modify the board or consume moves, score, or trigger cascades.
5. Reset the inactivity timer when the player interacts with the game, including mouse input and the R restart key.
6. Hide the hint as soon as the player interacts, starts a swap, or the game enters a win/loss state.

... 7. If no legal swap exists, do not display a false hint. Handle this case safely without the game or creating an infinite loop.

Show more Worked for 52s Tasks 1–3: invalid swaps do not consume moves, cascade scoring works, and special line-clear gems retain their behavior.

9. Keep the existing 8x8 grid, controls, score target, move budget, animations, and overlays.

10. Do not rewrite unrelated code.

After implementation:

- board.py
- game_engine.py
- Explain how idle time is tracked and how legal hint pairs are found.
- Identify the files changed.

Implementation:

- Run relevant automated checks if available and report the actual results.
- Provide manual test steps for the five-second delay, hint animation, input resetting the timer, and win/loss states.
- After more than 5 seconds, searches adjacent right/down pairs for a legal swap.
- Confirm how the implementation avoids mutating the board during hint detection.
- Hint detection temporarily swaps grid references, calls existing find_matches(), then restores them in a finally block.

Stop after Task 4. Do not claim tests passed unless they were actually run.

- No gems, scores, moves, animations, or cascades are changed during hint detection.
- Hints pulse with a yellow border around both suggested gems.
- Hints are cleared when:
 - Mouse input occurs
 - A swap begins
 - R resets the game
 - Animations are active
 - The game reaches win/loss state

- If no legal swap exists, `find_hint_swap()` returns `None` and the search is not repeatedly retried every frame.
- Existing Tasks 1–3 behavior remains intact.

Actual tests passed:

- Five-second timeout finds and renders a legal hint.
- Hint detection leaves the board unchanged.
- Mouse input hides the hint and resets the timer.
- Restart clears hint state and resets the timer.
- Game-over state hides the hint.
- Board with no legal swaps returns no hint safely.
- Task 1 invalid-swap regression passed.
- Task 2 two-wave cascade regression passed.
- Task 3 special-gem regression passed.
- Python compilation passed for `main.py`, `board.py`, and `game_engine.py`.

Manual test:

</>

powershell



`cd "Lab-04" python main.py`

1. Leave the game idle for over five seconds. A legal adjacent pair should pulse.
2. Confirm the highlighted swap creates a match.
3. Click anywhere or start a swap; the hint should disappear and the timer should restart.
4. Press R; the board should reset and no hint should remain.
5. Reach the win target or exhaust moves; the hint should disappear beneath the result overlay.
6. If the board has no legal swaps, no hint should be displayed.

Task 4 is complete; no further task was implemented.



Edited 2 files+67-1

Lab-04/game/board.py+43-0

Lab-04/game/game_engine.py+24-1

